Modern Java: Streams, Concurrency, JDBC & Patterns

Java 8 → 17 features, threads, streams, persistence, design patterns

10 topics · 88 flashcards · 52 quiz questions · 43 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Day 3: Modern Java: Streams, Concurrency, JDBC & Patterns

Java 8 → 17 features, threads, streams, persistence, design patterns

10 topics · 88 flashcards · 52 quiz questions · ~7.0 hours

> **Goal.** Walk in able to write a stream pipeline, explain synchronized vs volatile, and describe how a thread pool works. These three things alone cover most 'advanced Java' rounds.

Schedule

- Slot 1 (2h): Lambdas + Streams API (do every exercise)
- Slot 2 (1.5h): Optional + Date/Time API + Java 9–17 features
- Slot 3 (2.5h): Threads, synchronization, volatile, deadlock, wait/notify
- Slot 4 (2h): ExecutorService, concurrent collections, atomics
- Slot 5 (2h): JDBC, File I/O, serialization, annotations, design patterns

1. Lambdas & Method References Must-know 35 min

A lambda is an anonymous function you can pass around — but technically it's an **instance of a functional interface**. That distinction is what interviewers probe.

Key points

- Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`.
- A lambda can only target a **functional interface** (exactly one abstract method).
- Type inference: you can omit parameter types. One param with inferred type → parentheses optional (`s -> s.length()`).
- Lambdas can **capture** local variables, but only if they're **final or effectively final** (never reassigned).
- `this` inside a lambda refers to the **enclosing class** instance — unlike an anonymous inner class, where it refers to that class.
- **Method references**: `Class::staticMethod`, `object::instanceMethod`, `Class::instanceMethod`, `Class::new`.

Code

Lambda shapes, from verbose to minimal

```
// Anonymous inner class (the old way)
Comparator<String> oldWay = new Comparator<String>() {
   @Override public int compare(String a, String b) { return a.length() - b.length(); }
};

// Lambda, full syntax
Comparator<String> l1 = (String a, String b) -> { return Integer.compare(a.length(),
   b.length()); };

// Inferred types
Comparator<String> l2 = (a, b) -> Integer.compare(a.length(), b.length());

// Single expression -> no braces, no return
Comparator<String> l3 = (a, b) -> Integer.compare(a.length(), b.length());

// Method reference - because the body just delegates to an existing method
Comparator<String> l4 = Comparator.comparingInt(String::length);

// Runnable with no params
Runnable r = () -> System.out.println("running");
new Thread(() -> System.out.println("in a thread")).start();
```

Effectively final — the rule that trips everyone up

```
int factor = 2;                    // effectively final (never reassigned)
Function<Integer,Integer> times = n -> n * factor;   // OK: captures factor

// factor = 3;                     // uncomment -> COMPILE ERROR in the lambda above
// Local variables referenced from a lambda must be final or effectively final.
// Why? The lambda may outlive the stack frame, so Java copies the VALUE.

List<Integer> nums = new ArrayList<>();
Runnable add = () -> nums.add(1);  // OK - mutating the object is fine,
                                   // only REASSIGNING the reference is banned
```

The four kinds of method reference

```
// 1. Static method:        ClassName::staticMethod
Function<String, Integer> parser = Integer::parseInt;

// 2. Instance method of a particular object:  instance::method
String prefix = ">> ";
Function<String, String> addPrefix = prefix::concat;

// 3. Instance method of an arbitrary object of a given type: ClassName::method
Function<String, Integer> len   = String::length;      // s -> s.length()
BiFunction<String,String,Boolean> eq = String::equals; // (a,b) -> a.equals(b)

// 4. Constructor reference: ClassName::new
Supplier<List<String>> listFactory = ArrayList::new;
Function<String, BigDecimal> bdFactory = BigDecimal::new;

// Where you'll use them most: streams and sorting
list.stream().map(String::trim).filter(s -> !s.isEmpty()).forEach(System.out::println);
employees.sort(Comparator.comparing(Employee::getName));
```

this in a lambda vs an anonymous class

```
class Demo {
   String name = "Demo";
   Runnable asAnonymous() {
       return new Runnable() {
           String name = "Anonymous";
           public void run() { System.out.println(this.name); }  // "Anonymous"
       };
   }
   Runnable asLambda() {
       return () -> System.out.println(this.name);  // "Demo" - enclosing instance
   }
}
```

Output:

```
Anonymous
Demo
```

SAY IT LIKE THIS IN THE INTERVIEW

"A lambda is a concise instance of a functional interface — an interface with exactly one abstract method. Parameters types are inferred, and it can capture local variables as long as they're effectively final, because those values are copied. Method references are just shorthand when the lambda body only calls an existing method."

Flashcards

Q. What is a lambda expression in Java?
A. A concise way to provide an implementation of a functional interface — essentially an anonymous function passed as data.

Q. What must be true about the target type of a lambda?
A. It must be a functional interface: exactly one abstract method (default/static methods don't count).

Q. What does 'effectively final' mean?
A. A local variable that isn't declared final but is never reassigned after initialisation — lambdas and anonymous classes may capture those (and only those).

Q. Why must captured variables be effectively final?
A. The lambda may outlive the stack frame, so Java copies the value. If it could change, the copy would silently diverge from the original.

Q. What does `this` refer to inside a lambda?
A. The enclosing class instance — lambdas don't introduce a new scope. (Anonymous inner classes have their own `this`.)
Q. Name the four types of method reference.
A. ClassName::staticMethod, object::instanceMethod, ClassName::instanceMethod, ClassName::new.

Q. Can a lambda declare a checked exception?
A. Only if the functional interface's method declares it (e.g. Callable.call throws Exception). Otherwise you must handle it inside the body.

Quiz

Q1. Which of these is a valid functional interface target for a lambda?

[ ] A. interface A { void m(); void n(); }
[ ] B. abstract class B { abstract void m(); }
[x] C. interface C { int m(String s); }
[ ] D. interface D { }

> **Why:** C has exactly one abstract method. A has two, B is a class (not an interface), and D is a marker interface with none — none can be a lambda target.

Q2. What happens if you reassign a local variable after a lambda captures it?

[ ] A. Nothing — it just works
[x] B. Compile error: local variables referenced from a lambda must be final or effectively final
[ ] C. The lambda uses the new value
[ ] D. Runtime exception

> **Why:** Reassignment makes the variable non-final, and the compiler rejects the capture. Wrap mutable state in an object (or an AtomicInteger) instead.

Q3. `String::length` is which kind of method reference?

[ ] A. Static method reference
[ ] B. Instance method of a particular object
[x] C. Instance method of an arbitrary object of the type (s -> s.length())
[ ] D. Constructor reference

> **Why:** For `ClassName::instanceMethod`, the first parameter becomes the receiver: `(String s) -> s.length()`.

Q4. Is a lambda an object?

[ ] A. No, it's pure syntax sugar with no object
[x] B. Yes — it compiles to an instance of the functional interface (invokedynamic + a synthetic class)
[ ] C. It's a method pointer like in C
[ ] D. It's a String that gets evaluated

> **Why:** The JVM uses invokedynamic with a LambdaMetafactory to produce an instance of the target interface. Lambdas without captures are cached and reused.

2. Streams API Must-know 60 min

Streams are the single most practical Java 8 skill for interviews — they turn 15 lines of loop into 2, and they're a favourite 'refactor this code' exercise. Learn the pipeline shape and the top ~15 operations.

Key points

- A stream is **not a data structure** — it's a pipeline over a source. It doesn't store elements.
- Three parts: **source** → **intermediate operations** (lazy, return a Stream) → **terminal operation** (eager, produces a result).
- **Laziness**: nothing runs until a terminal operation is called (that's why a stream without one silently does nothing).
- Streams are **consumed once** — reusing a stream throws `IllegalStateException: stream has already been operated upon or closed`.
- Streams don't modify the source collection; they produce a new result.
- `parallelStream()` splits work across the ForkJoinPool — great for CPU-bound work on large data, bad for small collections, I/O, or anything with side effects / ordering requirements.

Reference

Category	Operations	Returns
Source	stream(), stream().of(), Arrays.stream(), Files.lines(), IntStream.range()	Stream
Filter / slice	filter, distinct, limit, skip, takeWhile, dropWhile	Stream
Map / flat	map, flatMap, mapToInt/Long/Double	Stream / IntStream
Sort / peek	sorted, sorted(comparator), peek (debug only)	Stream
Match (short-circuit)	anyMatch, allMatch, noneMatch, findFirst, findAny	boolean / Optional
Reduce	reduce, count, min, max, sum	Optional / value
Collect	collect(toList/toSet/toMap/groupingBy/joining)	Collection / Map / String

Code

Anatomy of a pipeline

```
List<Employee> emps = getEmployees();

List<String> result = emps.stream()                 // 1. SOURCE
   .filter(e -> e.getSalary() > 50000)            // 2. INTERMEDIATE (lazy)
   .sorted(Comparator.comparing(Employee::getSalary).reversed())
   .map(Employee::getName)                          //    transform
   .limit(10)
   .toList();                                       // 3. TERMINAL (triggers work)

// Nothing above executes until toList() is called - that's laziness.
// The source list is NEVER modified.
```

The 15 operations you'll actually use — with outputs

```
List<Integer> nums = List.of(5, 2, 9, 1, 5, 6, 3, 8);

nums.stream().filter(n -> n % 2 == 0).toList();          // [2, 6, 8]
nums.stream().map(n -> n * n).toList();                  // [25, 4, 81, 1, ...]
nums.stream().distinct().toList();                       // [5, 2, 9, 1, 6, 3, 8]
nums.stream().sorted().toList();                         // [1, 2, 3, 5, 5, 6, 8, 9]
nums.stream().sorted(Comparator.reverseOrder()).toList();// [9, 8, 6, 5, 5, 3, 2, 1]
nums.stream().limit(3).toList();                         // [5, 2, 9]
nums.stream().skip(2).toList();                          // [9, 1, 5, 6, 3, 8]

nums.stream().anyMatch(n -> n > 8);                      // true
nums.stream().allMatch(n -> n > 0);                      // true
nums.stream().noneMatch(n -> n < 0);                     // true
nums.stream().findFirst();                               // Optional[5]
nums.stream().count();                                   // 8
nums.stream().reduce(0, Integer::sum);                   // 39
nums.stream().mapToInt(Integer::intValue).sum();         // 39
nums.stream().max(Integer::compareTo);                   // Optional[9]

// flatMap: flatten nested collections (List<List<String>> -> Stream<String>)
List<List<String>> nested = List.of(List.of("a","b"), List.of("c"));
nested.stream().flatMap(List::stream).toList();          // [a, b, c]

// takeWhile / dropWhile (Java 9) on an ordered stream
List.of(1,2,3,4,1).stream().takeWhile(n -> n < 4).toList(); // [1,2,3]
```

Collectors — the terminal operations that build results

```
List<Employee> emps = getEmployees();

// toList / toSet / toCollection
emps.stream().map(Employee::getName).collect(Collectors.toList());
emps.stream().map(Employee::getName).collect(Collectors.toSet());   // dedupe

// joining - build a CSV
emps.stream().map(Employee::getName).collect(Collectors.joining(", "));
// "Parvez, Anil, Sneha"
Collectors.joining(", ", "[", "]");   // "[Parvez, Anil, Sneha]"

// groupingBy - the most powerful one
Map<String, List<Employee>> byDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment));

Map<String, Long> countByDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment,
                                               Collectors.counting()));

Map<String, Double> avgSalaryByDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment,
                            Collectors.averagingDouble(Employee::getSalary)));

// partitioningBy - always 2 groups (true/false)
Map<Boolean, List<Employee>> rich =
   emps.stream().collect(Collectors.partitioningBy(e -> e.getSalary() > 100000));

// toMap - watch out for duplicate keys!
Map<Integer, String> idToName = emps.stream()
   .collect(Collectors.toMap(Employee::getId, Employee::getName,
                             (a, b) -> a));    // <-- merge function avoids the crash

// summarizingInt - count, sum, min, average, max in one pass
IntSummaryStatistics stats =
   emps.stream().collect(Collectors.summarizingInt(Employee::getSalary));
stats.getAverage(); stats.getMax(); stats.getCount();
```

Laziness and short-circuiting — prove it with peek

```
Stream.of(1, 2, 3, 4, 5)
     .peek(n -> System.out.println("processing " + n))
     .filter(n -> n > 2)
     .findFirst();
// Output: processing 1 / processing 2 / processing 3
// It STOPS at 3 because findFirst short-circuits - elements 4 and 5 are never
// touched. With an infinite stream this is the difference between terminating
// and hanging:

Stream.iterate(0, n -> n + 1)          // infinite source - fine, because
     .filter(n -> n > 100)            // ...the pipeline is lazy and
     .findFirst()                     // ...findFirst short-circuits
     .orElse(-1);                     // 101

// Without a terminal operation, peek prints NOTHING.
```

Output:

```
processing 1
processing 2
processing 3
```

Refactoring exercise: before and after (a real interview task)

```
// BEFORE: find the names of the 3 highest-paid engineers, sorted alphabetically
List<String> result = new ArrayList<>();
for (Employee e : employees) {
   if ("Engineering".equals(e.getDepartment()) && e.getSalary() > 60000) {
       result.add(e.getName().toUpperCase());
   }
}
Collections.sort(result);
if (result.size() > 3) result = result.subList(result.size() - 3, result.size());

// AFTER
List<String> result2 = employees.stream()
   .filter(e -> "Engineering".equals(e.getDepartment()))
   .filter(e -> e.getSalary() > 60000)
   .sorted(Comparator.comparing(Employee::getSalary).reversed())
   .limit(3)
   .map(e -> e.getName().toUpperCase())
   .sorted()
   .toList();
```

Primitive streams and parallel streams

```
// Primitive streams avoid boxing overhead and add maths helpers
IntStream.range(1, 5).toArray();                 // [1, 2, 3, 4]  (end EXCLUSIVE)
IntStream.rangeClosed(1, 5).toArray();           // [1, 2, 3, 4, 5]
IntStream.rangeClosed(1, 100).sum();             // 5050
IntStream.rangeClosed(1, 100).average();         // OptionalDouble[50.5]
List.of(1,2,3).stream().mapToInt(Integer::intValue).sum();   // 6

// Parallel: splits across ForkJoinPool.commonPool()
long count = IntStream.rangeClosed(1, 1_000_000).parallel().filter(isPrime).count();

// Rules for parallelStream:
//  - Great for large, CPU-bound, stateless, order-independent work
//  - Bad for small collections (overhead > benefit), I/O, or shared mutable state
//  - Avoid if you depend on encounter order (unless you use forEachOrdered)
//  - Never use it with stateful lambdas or side effects
```

SAY IT LIKE THIS IN THE INTERVIEW

"A stream is a lazy pipeline over a data source: a source, zero or more intermediate operations like filter and map, and one terminal operation that triggers execution and produces a result. It doesn't store data or modify the source, and it can only be consumed once. I reach for Collectors like groupingBy and joining to build results, and I only use parallelStream for large CPU-bound work because the overhead and ordering

caveats usually aren't worth it."

Flashcards

Q. Is a stream a data structure?
A. No — it's a view/pipeline over a source (collection, array, generator). It doesn't store elements and never modifies the source.

Q. What's the difference between intermediate and terminal operations?
A. Intermediate (filter, map, sorted) are lazy and return a new Stream. Terminal (collect, forEach, count, reduce) are eager and trigger the whole pipeline.

Q. What happens if you use a stream twice?
A. IllegalStateException: stream has already been operated upon or closed. Create a new stream from the source instead.

Q. map vs flatMap?
A. map is 1→1 (each element becomes one element). flatMap is 1→many and flattens the resulting streams into one (List<List<T>> → Stream<T>).

Q. How do you avoid the duplicate-key crash in Collectors.toMap?
A. Pass a merge function as the third argument: `toMap(keyFn, valueFn, (a, b) -> a)`.

Q. groupingBy vs partitioningBy?
A. groupingBy groups by any key function (n groups). partitioningBy splits by a predicate into exactly two groups keyed true/false.

Q. When would you use parallelStream()?
A. For large, CPU-bound, stateless work where order doesn't matter. Avoid for small data, I/O, side effects or shared state — the ForkJoinPool overhead usually outweighs the gain.

Q. Why use IntStream instead of Stream<Integer>?
A. Avoids boxing/unboxing overhead and provides sum/average/min/max/summaryStatistics directly.

Q. Give the alphabetic order of a typical pipeline.
A. source → filter → map → sorted → distinct → limit → collect.

Q. What does peek() do and when is it appropriate?
A. Performs an action on each element as they flow through — intended for debugging/logging (e.g. proving laziness). Using it for real side effects is discouraged.

Quiz

Q1. What is the result of `Stream.of(1,2,3).filter(n -> n > 1);` with no terminal operation?

[ ] A. [2, 3]
[x] B. Nothing happens — intermediate operations are lazy
[ ] C. Compile error
[ ] D. An empty stream is stored

> **Why:** Without a terminal operation the pipeline is never executed. This also means any peek() logging prints nothing.

Q2. Which of these is a TERMINAL operation?

[ ] A. filter
[ ] B. map
[x] C. collect
[ ] D. sorted

> **Why:** collect, forEach, reduce, count, min, max, anyMatch, findFirst and toList() are terminal. filter/map/sorted/distinct/limit are intermediate.

Q3. `List<List<String>> nested; nested.stream().flatMap(List::stream).count()` — what does flatMap do here?

[ ] A. Counts the inner lists
[x] B. Flattens all inner lists into a single stream of Strings, then counts them

[ ] C. Throws a ClassCastException
[ ] D. Counts distinct lists

> **Why:** flatMap maps each element to a stream and concatenates them, turning Stream<List<String>> into Stream<String>.

Q4. Which Collector gives you a Map<String, Long> of counts per key?

[ ] A. Collectors.toMap
[x] B. Collectors.groupingBy(keyFn, Collectors.counting())
[ ] C. Collectors.counting()
[ ] D. Collectors.partitioningBy

> **Why:** groupingBy produces the map; the downstream collector counting() computes each group's size.

Q5. What is a risk of parallelStream()?

[ ] A. It always produces wrong ordering and is never usable
[x] B. Shared mutable state, ordering and ForkJoinPool overhead can make it slower or incorrect
[ ] C. It requires a special JVM flag
[ ] D. It can't be used with filter

> **Why:** Parallel streams share the common ForkJoinPool, and side effects/ordering bugs are easy to introduce. Use them only for big, independent, CPU-bound workloads.

Q6. `emps.stream().collect(Collectors.toMap(Employee::getDept, Employee::getName))` throws on duplicate departments. Why?

[ ] A. Because the value mapper is wrong
[x] B. toMap throws IllegalStateException on duplicate keys — you need a merge function
[ ] C. Because streams can't produce maps
[ ] D. Because getName returns null

> **Why:** Supply `(a, b) -> a` (or your own merge logic) as the third argument to handle collisions — or use groupingBy if you want all values per key.

3. Optional & the Date/Time API Must-know 35 min

Two Java 8 additions that come up constantly because they replaced bad old habits: `Optional` for nulls, and `java.time` for `Date`/`Calendar` (which were genuinely awful).

Key points

- `Optional<T>` is a **container** that may or may not hold a value — it's designed as a **return type** to force callers to handle absence.
- **Never** call `get()` without checking — use `orElse`, `orElseGet`, `orElseThrow`, `ifPresent`, `map`, `filter`.
- `orElse(x)` **always evaluates** x; `orElseGet(supplier)` only when empty. Use orElseGet for expensive defaults.
- Anti-patterns: using Optional as a field or parameter, `Optional.of(null)` (throws), `isPresent()+get()` (same as a null check).
- `java.time` is **immutable and thread-safe** — unlike `java.util.Date` and `Calendar`, which are mutable (a real bug source).
- Key types: `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant` (machine time), `Duration`, `Period`, `DateTimeFormatter`.

Reference

Need	Use	Example
Date only (birthday)	LocalDate	LocalDate.of(1999, 5, 21)
Date + time, no zone	LocalDateTime	LocalDateTime.now()
Date + time + zone	ZonedDateTime	ZonedDateTime.now(ZoneId.of("Asia/Kolkata"))
Timestamp / machine time	Instant	Instant.now() — great for DB/audit fields
Elapsed machine time	Duration	Duration.between(start, end)
Elapsed calendar time	Period	Period.between(d1, d2) → years/months/days
Parsing / formatting	DateTimeFormatter	DateTimeFormatter.ofPattern("dd-MM-yyyy")

Code

Optional done right (and wrong)

```
Optional<String> name = findNameById(42);

// WRONG
// if (name.isPresent()) System.out.println(name.get());   // just a null check
// Optional.of(maybeNull);        // throws NPE if null!
// Optional<String> field;        // don't use Optional as a field

// RIGHT
name.ifPresent(System.out::println);
name.ifPresentOrElse(System.out::println, () -> System.out.println("not found")); // Java 9

String value = name.orElse("default");
String lazy  = name.orElseGet(() -> expensiveLookup());   // only called if empty
String boom  = name.orElseThrow(() -> new NotFoundException("no name"));

// Transform without unwrapping
int len = name.map(String::length).orElse(0);
String upper = name.map(String::toUpperCase).filter(s -> s.length() > 3).orElse("");

// Chain nested nullables (replaces nested if-null checks)
String city = user.flatMap(User::getAddress).map(Address::getCity).orElse("Unknown");

// Java 9+: turn an Optional into a Stream
List<String> present = optionals.stream().flatMap(Optional::stream).toList();
```

Optional in real service code

```
// Repository returns Optional -> no null leaks
public Optional<User> findByEmail(String email) {
   return users.stream().filter(u -> u.getEmail().equals(email)).findFirst();
}

// Service: handle absence explicitly
public User getOrCreate(String email) {
   return findByEmail(email).orElseGet(() -> createUser(email));
}

// Controller: 404 instead of NPE
@GetMapping("/{id}")
public ResponseEntity<User> get(@PathVariable Long id) {
   return service.findById(id)
           .map(ResponseEntity::ok)
           .orElseGet(() -> ResponseEntity.notFound().build());
}
```

java.time in practice

```
// Creating
LocalDate today       = LocalDate.now();
LocalDate independence= LocalDate.of(1947, Month.AUGUST, 15);
LocalDateTime meeting = LocalDateTime.of(2026, 10, 10, 14, 30);
Instant timestamp     = Instant.now();                       // UTC, machine time
ZonedDateTime ist     = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));

// Arithmetic (immutable - always returns a NEW object)
LocalDate nextWeek    = today.plusWeeks(1);
LocalDate lastMonth   = today.minusMonths(1);
LocalDate firstOfMonth= today.withDayOfMonth(1);

// Comparing
today.isBefore(independence);   // false
today.isAfter(independence);    // true
ChronoUnit.DAYS.between(independence, today);   // long

// Period vs Duration
Period p  = Period.between(LocalDate.of(2020,1,1), LocalDate.of(2023,3,15)); // P3Y2M14D
Duration d = Duration.between(startTime, endTime);   // seconds/nanos
d.toMinutes();

// Formatting and parsing (DateTimeFormatter is IMMUTABLE and thread-safe)
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
String text = meeting.format(fmt);                    // "10-10-2026 14:30"
LocalDateTime parsed = LocalDateTime.parse("10-10-2026 14:30", fmt);

// Legacy interop (you'll need this with old JDBC code)
Date legacy     = Date.from(Instant.now());
Instant fromOld = legacy.toInstant();
```

Why java.util.Date was replaced

```
// OLD API problems:
Date d = new Date(2026, 9, 10);      // deprecated: year is 1900-based, month is 0-based!
d.setMonth(15);                      // MUTABLE - a shared Date gets corrupted
Calendar cal = Calendar.getInstance();
cal.set(Calendar.MONTH, Calendar.OCTOBER);   // 0-based months, verbose, mutable
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");  // NOT thread-safe!

// NEW API: clear, immutable, thread-safe, fluent
LocalDate clean = LocalDate.of(2026, 10, 10);      // real year, real month
DateTimeFormatter safe = DateTimeFormatter.ofPattern("dd-MM-yyyy");  // shareable
```

SAY IT LIKE THIS IN THE INTERVIEW

"Optional is a container that may hold a value, designed as a return type so absence is handled explicitly — I use orElse, orElseGet, orElseThrow and map/flatMap chains instead of get(), and I never use it as a field or parameter. For dates I use java.time: it's immutable and thread-safe, unlike Date and Calendar, and DateTimeFormatter can be shared across threads where SimpleDateFormat can't."

Flashcards

Q. What problem does Optional solve?
A. It makes absence explicit in the type system, forcing callers to handle the empty case instead of hitting NullPointerException.

Q. orElse vs orElseGet?
A. orElse takes a value that is ALWAYS evaluated; orElseGet takes a Supplier evaluated only when the Optional is empty. Use orElseGet for expensive defaults.

Q. Optional.of vs Optional.ofNullable?
A. of(null) throws NullPointerException; ofNullable(null) returns Optional.empty(). Use ofNullable when the value may be null.

Q. Should Optional be used as a field or method parameter?
A. No — it's designed as a return type. As a field it's not Serializable and adds overhead; as a parameter it forces callers to wrap values.

Q. Why is java.time better than java.util.Date?
A. java.time is immutable and thread-safe, has a clear API (LocalDate/LocalDateTime/Instant/ZonedDateTime), real month numbers, and a thread-safe DateTimeFormatter.

Q. LocalDate vs Instant?
A. LocalDate is a calendar date with no time or zone (a birthday). Instant is a machine timestamp on the UTC timeline (audit fields, DB timestamps).

Q. Is SimpleDateFormat thread-safe?
A. No — sharing a static instance across threads corrupts results. DateTimeFormatter in java.time IS immutable and thread-safe.

Q. Period vs Duration?
A. Period measures calendar time (years, months, days). Duration measures exact machine time (seconds, nanos).

Quiz

Q1. What's the difference between orElse(compute()) and orElseGet(() -> compute())?

[ ] A. No difference
[x] B. orElse always evaluates compute(); orElseGet only evaluates it when the Optional is empty
[ ] C. orElseGet is deprecated
[ ] D. orElse throws if empty

> **Why:** orElse's argument is evaluated eagerly at call time. That's a real performance bug when the default requires a database call — use orElseGet.

Q2. `Optional.of(null)` results in:

[ ] A. Optional.empty()
[x] B. NullPointerException
[ ] C. Compile error
[ ] D. A warning only

> **Why:** of() requires a non-null value and throws NPE immediately. Use ofNullable() when the value might be null.

Q3. Which is immutable and thread-safe?

[ ] A. java.util.Date
[ ] B. java.util.Calendar
[x] C. LocalDateTime
[ ] D. SimpleDateFormat

> **Why:** All java.time types are immutable and thread-safe. Date and Calendar are mutable; SimpleDateFormat is not thread-safe.

Q4. How do you get the number of days between two LocalDate objects?

[ ] A. d2 - d1
[x] B. ChronoUnit.DAYS.between(d1, d2)
[ ] C. d1.minus(d2).getDays()
[ ] D. Duration.between(d1, d2).toDays()

> **Why:** ChronoUnit.DAYS.between(a, b) is the idiomatic call. (Duration doesn't work on LocalDate because there's no time component.)

Q5. What is the recommended use of Optional in an API?

[ ] A. As a field type to avoid nulls in entities
[x] B. As a method return type to signal 'may be absent'
[ ] C. As a parameter type
[ ] D. In constructors

> **Why:** Optional is a return-type convention. Fields make the class non-Serializable-friendly and add indirection; parameters force awkward wrapping.

4. Modern Java: 9 → 17 Features Edge 35 min

Most fresher candidates only know Java 8. Dropping `records`, `sealed`, `var`, and switch expressions into an answer is a cheap way to look current — and Java 17 is the current LTS, so your interview is probably on 11 or 17.

Key points

- **Java 9**: module system (JPMS), `List.of/Map.of/Set.of`, `Optional.ifPresentOrElse`, `Stream.takeWhile/dropWhile/ofNullable/iterate`, private interface methods, JShell.
- **Java 10**: `var` local variable type inference (locals only, must be initialised — not for fields, parameters or return types).
- **Java 11 (LTS)**: `var` in lambda params, HTTP Client (java.net.http), String `isBlank/strip/lines/repeat/indent`, `Files.readString/writeString`, single-file source launch.
- **Java 14/15**: **switch expressions** (`->`, `yield`), helpful NPE messages, **text blocks**, **records** (preview).
- **Java 16**: **records** (final), **pattern matching for instanceof**, Stream.toList().
- **Java 17 (LTS)**: **sealed classes** (`permits`), pattern matching for switch (preview), strong encapsulation of JDK internals.

Reference

Feature	Version	What it gives you
var	10	Less boilerplate for obvious local types
Records	16	Immutable data carriers — equals/hashCode/toString/accessors for free
Sealed classes	17	Restrict which classes may extend/implement — enables exhaustive pattern matching
Switch expressions	14	switch returns a value, no fall-through, exhaustive
Text blocks	15	Readable multi-line strings (JSON, SQL, HTML)
Pattern matching instanceof	16	Test and cast in one step
HTTP Client	11	Modern sync/async HTTP with HTTP/2 and WebSocket

Code

Records — the DTO killer (Java 16+)

```
// ONE line replaces a class with fields, getters, equals, hashCode, toString
public record UserDto(Long id, String name, String email) { }

UserDto u = new UserDto(1L, "Parvez", "p@example.com");
u.name();            // "Parvez"  (accessor, NOT getName())
u.equals(new UserDto(1L, "Parvez", "p@example.com"));   // true - value equality!
System.out.println(u);   // UserDto[id=1, name=Parvez, email=p@example.com]

// Records are implicitly final, and their fields are private final.
// You can still add methods, compact constructors and validation:
public record Range(int start, int end) {
   public Range {                       // compact constructor - validation
       if (start > end) throw new IllegalArgumentException("start > end");
   }
   public int length() { return end - start; }
}

// Where you use them: DTOs, API responses, Map keys, multi-value returns,
// and anywhere you'd otherwise write a Lombok @Data class.
```

Output:
```
UserDto[id=1, name=Parvez, email=p@example.com]
```

Sealed classes — controlled inheritance (Java 17+)

```
// Only the listed classes may extend Shape. Nothing else can.
public sealed interface Shape permits Circle, Square, Rectangle { }

record Circle(double radius) implements Shape { }
record Square(double side)   implements Shape { }
final class Rectangle implements Shape { }   // permitted subclasses must be
                                             // final, sealed, or non-sealed

// Because the hierarchy is closed, the compiler can prove exhaustiveness:
double area(Shape s) {
   return switch (s) {                    // NO default needed!
       case Circle c    -> Math.PI * c.radius() * c.radius();
       case Square sq   -> sq.side() * sq.side();
       case Rectangle r -> r.width() * r.height();
   };
}
```

Pattern matching, var, and text blocks

```
// instanceof + cast, OLD:
if (obj instanceof String) {
   String s = (String) obj;
   System.out.println(s.length());
}
// NEW (Java 16) - test and bind in one step:
if (obj instanceof String s && s.length() > 3) {
   System.out.println(s.toUpperCase());     // s is in scope here
}

// var - local type inference (Java 10)
var list = new ArrayList<String>();     // ArrayList<String>
var entry = map.entrySet().iterator().next();
// var x;          -> ERROR: needs an initialiser
// var x = null;   -> ERROR: can't infer
// Use it when the type is OBVIOUS from the right-hand side; otherwise spell
// it out - readability beats brevity.

// Text blocks (Java 15)
String query = """
   SELECT u.name, u.email
   FROM users u
   WHERE u.active = true
   ORDER BY u.name
   """;
```

Small but high-value additions

```
// Java 9: convenient immutable collections
List<String> l = List.of("a", "b");         // immutable, no nulls
Map<String,Integer> m = Map.of("a", 1, "b", 2);
Map<String,Integer> m5 = Map.ofEntries(Map.entry("a", 1), Map.entry("b", 2)); // >10 pairs
Set<Integer> s = Set.of(1, 2, 3);

// Java 9: Stream improvements
Stream.ofNullable(maybeNull);                       // empty stream if null
Stream.iterate(1, n -> n < 100, n -> n * 2).toList();  // has a predicate
List.of(1,2,3,4,5).stream().takeWhile(n -> n < 4).toList();  // [1,2,3]

// Java 11: String helpers
"  ".isBlank();          // true
"  hi  ".strip();        // "hi"    (Unicode-aware)
"hi\nthere".lines().toList();  // [hi, there]
"ab".repeat(3);          // "ababab"

// Java 11: run a single file without compiling
// java Hello.java

// Java 11: HTTP Client
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder()
       .uri(URI.create("https://api.example.com/users"))
       .header("Accept", "application/json")
       .GET().build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode() + " " + res.body());
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'm comfortable on Java 11 and 17, the current LTS versions. Beyond Java 8 I use records for immutable DTOs instead of Lombok boilerplate, sealed interfaces with pattern-matching switch for closed hierarchies, var where the type is obvious, text blocks for SQL and JSON, and the Java 11 HTTP Client instead of HttpURLConnection."

Flashcards

Q. What is a record and when do you use it?
A. A transparent, immutable data carrier (Java 16): `record Point(int x, int y)`. The compiler generates a canonical constructor, accessors, equals, hashCode and toString. Use it for DTOs, API models and Map keys.

Q. What is a sealed class/interface?
A. Java 17: `sealed interface Shape permits Circle, Square` restricts which types may implement it. Permitted subclasses must be final, sealed or non-sealed — enabling exhaustiveness checks.

Q. Where can you use var?
A. Only for local variables with an initialiser (and lambda parameters since Java 11). Not fields, method parameters, return types, or with null.

Q. What does `instanceof String s` do?
A. Pattern matching (Java 16): tests the type and, if it matches, declares and assigns the variable in one step, scoping it to where the match is guaranteed.

Q. What is a text block?
A. A multi-line string literal delimited by triple quotes (Java 15) that preserves formatting and strips incidental indentation — ideal for SQL, JSON and HTML.

Q. Which Java versions are LTS?
A. 8, 11, 17 and 21. Most enterprises run 11 or 17.

Q. What replaced HttpURLConnection?
A. The java.net.http.HttpClient (Java 11) — supports HTTP/1.1, HTTP/2, synchronous and asynchronous calls, and WebSockets.

Q. What is JShell?
A. The Java REPL (Java 9) for quickly experimenting with snippets: type `jshell` at the terminal.

Quiz

Q1. Which is true about records?

[ ] A. They're mutable JavaBeans
[x] B. They're immutable; accessors are named after the field (u.name(), not getName())
[ ] C. They can be extended by subclasses
[ ] D. They only exist at compile time

> **Why:** Records are implicitly final with private final fields. Accessors use the field name directly, and equals/hashCode/toString are generated with value semantics.

Q2. Why does a sealed interface let you omit `default` in a switch expression?

[ ] A. Because switch never needs a default
[x] B. Because the compiler knows the complete set of permitted subtypes and can prove exhaustiveness
[ ] C. Because sealed types can't be switched on
[ ] D. Because records never fail

> **Why:** That's the whole point of sealing: a closed hierarchy makes exhaustive pattern matching checkable at compile time.

Q3. Where can `var` NOT be used?

[ ] A. Local variables with an initialiser
[ ] B. Enhanced-for loop variables
[x] C. Method parameters and fields
[ ] D. try-with-resources

> **Why:** var is restricted to local variable declarations (and lambda params in Java 11+). Fields and method signatures must be explicit.

Q4. Which Java version introduced the module system (JPMS)?

[ ] A. Java 8
[x] B. Java 9
[ ] C. Java 11
[ ] D. Java 17

> **Why:** Java 9 introduced Project Jigsaw modules (module-info.java). It's mostly relevant for library and JDK internals work, not daily app code.

Q5. What is the output of `"Java".repeat(2) + "\u0021"`?

[x] A. JavaJava!
[ ] B. Java!Java!
[ ] C. Java2!
[ ] D. Compile error

> **Why:** repeat(n) (Java 11) concatenates the string n times, giving "JavaJava", then "!" — though note \u0021 is processed by the compiler as a unicode escape, which still yields !.

5. Threads: Creation, Lifecycle & Basics Must-know 45 min

Multithreading is where fresher interviews get serious. Start with the lifecycle diagram and the two ways to create a thread — everything else builds on that.

Key points

- **Process** = a running program with its own memory. **Thread** = a unit of execution inside a process; threads share heap memory but have their own stack.
- Two ways to create: `extends Thread` or `implements Runnable` (preferred — you can still extend another class, and it separates the task from the runner).
- **Always call `start()`, never `run()`** — `run()` just executes on the current thread, no new thread is created.
- **Lifecycle**: NEW → (start) → RUNNABLE ⇄ RUNNING → (wait/sleep/join) → WAITING/TIMED_WAITING/BLOCKED → TERMINATED.
- `sleep()` holds the lock and pauses; `wait()` **releases** the lock and must be called inside `synchronized`; `yield()` is only a hint; `join()` waits for the thread to finish.
- A **daemon thread** is a background service thread — the JVM exits when only daemon threads remain.

Reference

Method	Releases lock?	Purpose	Class
sleep(ms)	No	Pause the current thread for a duration	Thread (static)
wait()	Yes	Wait until notify/notifyAll — must be in synchronized	Object (final)
notify()/notifyAll()	On exit of the block	Wake one/all waiting threads	Object (final)
join()	No	Wait for another thread to die	Thread
yield()	No	Hint: let another thread run (no guarantee)	Thread (static)
interrupt()	-	Signal a thread to stop blocking/stop work	Thread

Code

Two ways to create a thread — and the start() vs run() trap

```
// Way 1: extend Thread (simple, but you've used up your one inheritance)
class MyThread extends Thread {
   @Override public void run() { System.out.println("running in " + getName()); }
}
new MyThread().start();

// Way 2: implement Runnable (PREFERRED) - task decoupled from the thread
Runnable task = () -> System.out.println("running in " + Thread.currentThread().getName());
new Thread(task).start();
new Thread(task, "worker-1").start();     // named threads are easier to debug

// Way 3: Callable + Future when you need a RESULT or checked exceptions
ExecutorService pool = Executors.newSingleThreadExecutor();
Future<Integer> f = pool.submit(() -> 21 + 21);
Integer result = f.get();     // blocks until done -> 42

// THE TRAP
Thread t = new Thread(() -> System.out.println("hi"));
t.run();     // WRONG: runs on the MAIN thread, no concurrency
t.start();   // RIGHT: the JVM creates a new thread and calls run() there

// Calling start() twice -> IllegalThreadStateException
```

Thread lifecycle and states

```
//   new Thread()        --> NEW
//        .start()       --> RUNNABLE (waiting for the scheduler)
//   scheduled           --> RUNNING
//        .sleep(n) / .wait(timeout) / .join(timeout)  --> TIMED_WAITING
//        .wait()  / .join()                           --> WAITING
//   trying to enter a synchronized block held by another thread --> BLOCKED
//   run() returns       --> TERMINATED

Thread t = new Thread(() -> {
   try { Thread.sleep(1000); } catch (InterruptedException e) {
       Thread.currentThread().interrupt();   // restore the interrupt flag
   }
});
t.start();
System.out.println(t.getState());   // RUNNABLE or TIMED_WAITING
t.join();                           // main waits here until t finishes
System.out.println(t.getState());   // TERMINATED

// Daemon threads: JVM exits when only daemons remain
Thread daemon = new Thread(() -> { while (true) { /* background work */ } });
daemon.setDaemon(true);             // MUST be set before start()
daemon.start();
```

sleep vs wait — the #1 multithreading question

```
// sleep: static, on Thread, keeps the lock, wakes after the timeout
synchronized (lock) {
   Thread.sleep(1000);   // still HOLDS the lock - others stay blocked
}

// wait: on Object, releases the lock, needs notify to wake
synchronized (lock) {
   lock.wait();          // RELEASES the lock, other threads can enter
   // ... after notify() + re-acquiring the lock, continues here
}
// Another thread:
synchronized (lock) { lock.notify(); }   // or notifyAll()

// Rules:
//  - wait/notify/notifyAll must be called from INSIDE a synchronized block
//    on the same object -> IllegalMonitorStateException otherwise
//  - Always wait() in a LOOP (spurious wakeups are allowed):
synchronized (lock) {
   while (!condition) { lock.wait(); }
}
//  - Prefer notifyAll() over notify() unless you can prove one waiter suffices.
```

Race condition — see the bug before you fix it

```
class Counter {
   private int count = 0;
   public void increment() { count++; }   // NOT atomic! read -> add -> write
   public int get() { return count; }
}

Counter c = new Counter();
Runnable r = () -> { for (int i = 0; i < 10_000; i++) c.increment(); };
Thread t1 = new Thread(r), t2 = new Thread(r);
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println(c.get());   // Expected 20000, actual ~14000 (varies!)

// Fixes, in order of preference:
// 1. AtomicInteger   -> private final AtomicInteger c = new AtomicInteger();
// 2. synchronized    -> public synchronized void increment() { count++; }
// 3. Lock            -> lock.lock(); try { count++; } finally { lock.unlock(); }
// 4. LongAdder       -> best under very high contention
```

```
Output:
~14000 (non-deterministic)
```

SAY IT LIKE THIS IN THE INTERVIEW

"A thread is a lightweight unit of execution inside a process; threads share heap memory but each has its own stack. I implement Runnable and call start() — never run() directly, which would just execute on the calling thread. For lifecycle: NEW, RUNNABLE, RUNNING, BLOCKED/WAITING/TIMED_WAITING, TERMINATED. And the key difference: sleep() keeps the lock, wait() releases it and must be called inside a synchronized block."

Flashcards

Q. Thread vs Runnable — which should you use and why?
A. Implement Runnable. Java has single inheritance, so extending Thread wastes your one superclass, and Runnable separates the task from the thread running it (so it also works with executors).

Q. What happens if you call run() instead of start()?
A. No new thread is created — run() executes synchronously on the current thread. Only start() creates a thread; calling start() twice throws IllegalThreadStateException.

Q. sleep() vs wait()?
A. sleep is a Thread static method that pauses without releasing the lock. wait is an Object method that releases the monitor and must be called inside synchronized; it needs notify/notifyAll to resume.

Q. Why must wait() be inside a synchronized block?
A. Because wait() releases the monitor the thread owns — if it doesn't own it, you get IllegalMonitorStateException. Reacquisition is also required on wake.

Q. What is a daemon thread?
A. A background/service thread. The JVM terminates when only daemon threads remain — used for GC and background housekeeping.

Q. What is a race condition?
A. When the result depends on the unpredictable interleaving of threads accessing shared mutable state — e.g. count++ (read-modify-write) losing updates.

Q. Why should wait() be called in a loop?
A. Because of spurious wakeups (the JVM may wake a thread without notify) and because the condition may still be false when the lock is reacquired.

Q. What is a deadlock?
A. Two or more threads each hold a lock the other needs, so all block forever. Prevent it with consistent lock ordering, timeouts (tryLock), or by avoiding nested locks.

Q. notify vs notifyAll?
A. notify wakes one arbitrary waiting thread; notifyAll wakes all of them (they then compete for the lock). Prefer notifyAll unless exactly one waiter is guaranteed.

Quiz

Q1. You call `thread.run()` instead of `thread.start()`. What happens?

[ ] A. A new thread runs the code
[x] B. The code runs on the current (calling) thread — no concurrency
[ ] C. Compile error
[ ] D. IllegalThreadStateException

> **Why:** run() is just a method call. start() is what asks the JVM to spawn a new thread and then invoke run() on it.

Q2. Which releases the object's monitor lock?

[ ] A. Thread.sleep(100)
[x] B. obj.wait()
[ ] C. Thread.yield()
[ ] D. t.join()

> **Why:** wait() releases the monitor so other threads can enter the synchronized block. sleep, yield and join all hold the lock.

Q3. What is the thread state after calling start() but before the OS schedules it?

[ ] A. NEW
[x] B. RUNNABLE
[ ] C. RUNNING
[ ] D. BLOCKED

> **Why:** start() moves NEW → RUNNABLE, meaning 'ready to run and waiting for CPU'. Java's Thread.State has no separate RUNNING state.

Q4. What exception do you get from wait() outside a synchronized block?

[ ] A. InterruptedException
[x] B. IllegalMonitorStateException
[ ] C. IllegalStateException
[ ] D. No exception

> **Why:** The thread doesn't own the object's monitor, so wait() (and notify) throw IllegalMonitorStateException.

Q5. How do you correctly stop a running thread in modern Java?

[ ] A. Thread.stop()
[x] B. Cooperatively: set a volatile flag or call interrupt(), and let the thread exit its loop

[ ] C. System.exit(0)
[ ] D. Thread.suspend()

Why: stop(), suspend() and resume() are deprecated and unsafe (they leave objects half-updated). Use interrupt() plus a checked flag, or Future.cancel(true).

Q6. What is the output order guarantee when starting two threads t1 and t2?

[ ] A. t1 always finishes first
[x] B. No guarantee — thread scheduling is up to the JVM/OS
[ ] C. They run simultaneously and interleave deterministically
[ ] D. main() finishes first

Why: Never rely on thread ordering. Use join(), CountDownLatch, or an ExecutorService if you need coordination.

6. Synchronization, volatile & Deadlock Must-know 50 min

The heart of Java concurrency interviews: `synchronized`, `volatile`, what each guarantees, and how to avoid deadlock. Learn the memory-model reason for volatile — most candidates can't explain it.

Key points

- `synchronized` gives **mutual exclusion** AND **visibility** (happens-before). It can be a method or a block; it locks on an object monitor (or the Class object for static methods).
- `synchronized` is **reentrant** — a thread holding the lock can enter other synchronized blocks on the same monitor (that's why a synchronized method can call another).
- `volatile` guarantees **visibility and ordering** but **not atomicity** — so `volatile int i; i++` is still not thread-safe.
- Use volatile for a **flag or a single-writer value** (e.g. a stop flag, a safely-published config reference) where the write doesn't depend on the current value.
- **Deadlock** needs four conditions: mutual exclusion, hold-and-wait, no preemption, circular wait. Break any one — usually circular wait, via consistent lock ordering.
- **Livelock**: threads keep reacting to each other but make no progress. **Starvation**: a thread never gets CPU/lock access (often priority-related).

Reference

Aspect	synchronized	volatile
Mutual exclusion	Yes	No
Visibility	Yes	Yes
Atomicity	Yes (for the whole block)	No — only single reads/writes
Blocking	Yes (thread blocks)	No (never blocks)
Reentrant	Yes	N/A
Applies to	Methods and blocks	Fields only
Compiler reordering	Prevents it across the boundary	Prevents it around the volatile access
Use for	Compound actions (check-then-act, read-modify-write)	Flags, status, one-time publication

Code

synchronized: method, block, static — what's the lock?

```
class Counter {
   private int a = 0, b = 0;
   private static int staticCount = 0;
   private final Object lock = new Object();

   public synchronized void incA() { a++; }      // locks on `this`
   public void incB() { synchronized (this) { b++; } }   // same lock

   public void incBoth() { synchronized (lock) { a++; b++; } } // private lock
   // ^ PREFERRED: a private lock object can't be locked by outside code,
   //   which prevents accidental deadlocks and broken encapsulation.

   public static synchronized void incStatic() { staticCount++; }
   // locks on Counter.class (the Class object), NOT on any instance
   // equivalent to: synchronized (Counter.class) { ... }
}
```

volatile: what it fixes — and what it doesn't

```
// WITHOUT volatile, this loop may NEVER terminate:
// the reading thread caches `running` in a register and never sees the update.
class Worker implements Runnable {
   private volatile boolean running = true;   // <-- volatile fixes visibility
   public void run() { while (running) { /* do work */ } }
   public void stop() { running = false; }    // another thread calls this
}

// What volatile does NOT do:
class Bad {
   private volatile int count = 0;
   public void increment() { count++; }   // STILL BROKEN
   // count++ = read, add, write - three steps. Two threads can read the same
   // value and both write count+1, losing an update.
   // Fix: AtomicInteger, synchronized, or LongAdder.
}

// Volatile IS enough when: one thread writes, others only read, and the new
// value doesn't depend on the old one (a flag, a published immutable object).
class Config {
   private volatile Map<String,String> settings = Map.of();  // safe publication
   void reload() { settings = loadFromDisk(); }   // atomic reference write
   String get(String k) { return settings.get(k); }
}
```

Deadlock: reproduce it, then fix it

```
// DEADLOCK: two locks acquired in opposite orders
Object lockA = new Object(), lockB = new Object();

Thread t1 = new Thread(() -> {
   synchronized (lockA) {
       sleep(50);
       synchronized (lockB) { System.out.println("t1 got both"); }
   }
});
Thread t2 = new Thread(() -> {
   synchronized (lockB) {                 // <-- reverse order = circular wait
       sleep(50);
       synchronized (lockA) { System.out.println("t2 got both"); }
   }
});
t1.start(); t2.start();   // hangs forever

// FIX 1 (best): always acquire locks in a CONSISTENT global order (A then B).
// FIX 2: use ReentrantLock with tryLock + timeout
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
   try {
       if (lockB.tryLock(1, TimeUnit.SECONDS)) {
           try { /* critical section */ } finally { lockB.unlock(); }
       }
   } finally { lockA.unlock(); }
}
// FIX 3: avoid nested locking altogether - restructure so one lock suffices.
// FIX 4: detect it - `jstack <pid>` prints "Found one Java-level deadlock".
```

Producer–Consumer with wait/notifyAll (a common whiteboard task)

```
class BlockingBuffer {
   private final Queue<Integer> queue = new LinkedList<>();
   private final int CAPACITY = 5;

   public synchronized void produce(int v) throws InterruptedException {
       while (queue.size() == CAPACITY) wait();   // loop, not if!
       queue.add(v);
       notifyAll();                               // wake the consumers
   }

   public synchronized int consume() throws InterruptedException {
       while (queue.isEmpty()) wait();
       int v = queue.remove();
       notifyAll();                               // wake the producers
       return v;
   }
}
// In real code you'd use BlockingQueue instead:
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(5);
q.put(1);        // blocks when full
int v = q.take();// blocks when empty - no wait/notify needed
```

ReentrantLock, ReadWriteLock and AtomicInteger

```
// ReentrantLock - more flexible than synchronized
private final ReentrantLock lock = new ReentrantLock(true);  // true = fair
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); }        // ALWAYS in finally

// tryLock avoids blocking forever
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) { try { ... } finally { lock.unlock(); } }

// ReadWriteLock: many concurrent readers, exclusive writer
private final ReadWriteLock rw = new ReentrantReadWriteLock();
private final Map<String,String> cache = new HashMap<>();
String read(String k) {
   rw.readLock().lock(); try { return cache.get(k); } finally { rw.readLock().unlock(); }
}
void write(String k, String v) {
   rw.writeLock().lock(); try { cache.put(k, v); } finally { rw.writeLock().unlock(); }
}

// AtomicInteger - lock-free, CAS-based
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();            // atomic ++
counter.addAndGet(5);
counter.compareAndSet(10, 20);        // CAS: set to 20 only if currently 10
AtomicReference<User> ref = new AtomicReference<>();
LongAdder adder = new LongAdder();    // faster than AtomicInteger under high contention
adder.increment(); adder.sum();
```

SAY IT LIKE THIS IN THE INTERVIEW

"synchronized provides both mutual exclusion and the visibility guarantees of happens-before, and it's reentrant. volatile gives visibility and prevents reordering but not atomicity, so I use it for flags and safe publication, never for compound operations like i++. For deadlock, the practical fix is consistent lock ordering, plus tryLock with a timeout; in production I'd diagnose with jstack or a profiler."

Flashcards

Q. What does synchronized guarantee?
A. Mutual exclusion (only one thread in the block) and visibility/happens-before (changes are flushed to main memory and seen by the next thread to acquire the lock).

Q. Is synchronized reentrant?
A. Yes — a thread that holds a monitor can re-enter any synchronized block on that same monitor, which prevents self-deadlock when synchronized methods call each other.

Q. What does volatile guarantee?
A. Visibility (writes are immediately visible to other threads), ordering (no reordering across the volatile access), and atomicity of single reads/writes — but NOT compound operations.

Q. Is `volatile int i; i++` thread-safe?
A. No. i++ is read-modify-write. Use AtomicInteger, synchronized, or LongAdder.

Q. What lock does a static synchronized method use?
A. The Class object (Counter.class), not an instance — so it's independent of any instance-level synchronized method.

Q. What are the four conditions for deadlock?
A. Mutual exclusion, hold-and-wait, no preemption, circular wait. Break any one (usually circular wait via consistent ordering) to prevent deadlock.

Q. Deadlock vs livelock vs starvation?
A. Deadlock: blocked forever holding locks. Livelock: threads keep responding to each other but make no progress. Starvation: a thread never gets resources.

Q. synchronized vs ReentrantLock?
A. ReentrantLock adds tryLock with timeout, fairness policy, interruptible locking and multiple conditions — but you must unlock in a finally block. Prefer synchronized for simplicity unless you need those features.

Q. How do you detect a deadlock in production?
A. Take a thread dump: `jstack <pid>` (it reports 'Found one Java-level deadlock'), or use JConsole/VisualVM/YourKit.

Q. How does AtomicInteger work?
A. It uses CAS (compare-and-swap) CPU instructions in a loop — lock-free and generally faster than synchronized for simple counters.

Quiz

Q1. Which of these is thread-safe?

[ ] A. private volatile int count; void inc(){ count++; }
[x] B. private int count; synchronized void inc(){ count++; }
[ ] C. private int count; void inc(){ count++; }
[ ] D. private static int count; void inc(){ count++; }

> **Why:** Only synchronized makes read-modify-write atomic. volatile alone doesn't — that's exactly the misconception to avoid.

Q2. What object does `synchronized` lock on in an instance method?

[ ] A. The class object
[x] B. `this` (the current instance)
[ ] C. A new Object each time
[ ] D. The method itself

> **Why:** An instance synchronized method locks the receiver (`this`). A static synchronized method locks the Class object.

Q3. How do you prevent deadlock when two locks are needed?

[ ] A. Use more threads
[x] B. Always acquire locks in the same global order (and consider tryLock with a timeout)
[ ] C. Make all methods synchronized
[ ] D. Use volatile instead

> **Why:** Consistent acquisition order removes the circular-wait condition. tryLock with a timeout adds a safety net; a private lock object prevents external interference.

Q4. What is the difference between notify() and notifyAll()?

[ ] A. notify wakes all, notifyAll wakes one
[x] B. notify wakes one arbitrary waiting thread; notifyAll wakes all waiters
[ ] C. They're identical

[ ] D. notifyAll is deprecated

> **Why:** notify() picks one arbitrary waiter, which can deadlock if the wrong thread wakes. notifyAll is safer; default to it.

Q5. Does making a field volatile make all operations on it atomic?

[ ] A. Yes
[x] B. No — only single reads and writes; compound operations still need synchronization
[ ] C. Yes, for ints only
[ ] D. Only in synchronized methods

> **Why:** volatile = visibility + ordering. Atomicity of compound actions (i++, check-then-act) requires synchronized, a Lock, or an atomic class.

Q6. Reentrancy means:

[x] A. A thread can acquire the same lock it already holds without deadlocking
[ ] B. Locks can be shared between threads
[ ] C. A method can be called recursively without a stack overflow
[ ] D. Locks are automatically released

> **Why:** Java monitors are reentrant: the JVM tracks the owner and a hold count, so a thread can re-enter any synchronized block it already owns.

7. ExecutorService & Concurrent Collections Must-know 45 min

"Don't create threads manually — use a pool" is the modern answer, and interviewers check that you know why, plus which pool to pick.

Key points

- Creating threads is expensive (each reserves ~1MB of stack) and unbounded threads exhaust memory. A **pool reuses** a fixed number of threads.
- `Executors.newFixedThreadPool(n)` — bounded pool with an unbounded queue. `newCachedThreadPool()` — unbounded, dangerous under load. `newSingleThreadExecutor()` — sequential. `newScheduledThreadPool()` — delays/periodics.
- Prefer constructing `ThreadPoolExecutor` explicitly in production so you control the queue size and rejection policy (the Executors factory defaults to an unbounded queue → OOM risk).
- `submit()` returns a `Future`; `get()` blocks; `invokeAll` runs a batch; `shutdown()` stops accepting new work and lets queued tasks finish (`shutdownNow()` interrupts).
- `Callable<V>` returns a value and can throw checked exceptions; `Runnable` cannot.
- Concurrent collections: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` (ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue), `ConcurrentLinkedQueue`.

Reference

Tool	Use it for
ExecutorService	Managing a pool of worker threads instead of new Thread()
Future<T>	A handle on an async result (get, cancel, isDone)
CompletableFuture<T>	Composing async pipelines: thenApply, thenCompose, allOf
ConcurrentHashMap	Thread-safe map without locking the whole structure
BlockingQueue	Producer–consumer handoff (put/take block)
CountDownLatch	Wait for N tasks to finish (one-shot)
CyclicBarrier	Make N threads wait for each other at a barrier (reusable)
Semaphore	Limit concurrent access to a resource (permits)

Code

ExecutorService: the pattern you should write from memory

```
ExecutorService pool = Executors.newFixedThreadPool(4);

// Fire-and-forget
pool.execute(() -> System.out.println("task"));

// With a result
Future<Integer> f = pool.submit(() -> {
   TimeUnit.MILLISECONDS.sleep(200);
   return 42;
});
// ... do other work ...
Integer result = f.get();                 // blocks; throws ExecutionException
Integer safe   = f.get(1, TimeUnit.SECONDS);  // throws TimeoutException

// Batch
List<Callable<Integer>> tasks = List.of(() -> 1, () -> 2, () -> 3);
List<Future<Integer>> futures = pool.invokeAll(tasks);
for (Future<Integer> fu : futures) System.out.println(fu.get());

// ALWAYS shut down - otherwise the JVM never exits
pool.shutdown();
try {
   if (!pool.awaitTermination(60, TimeUnit.SECONDS)) pool.shutdownNow();
} catch (InterruptedException e) { pool.shutdownNow(); Thread.currentThread().interrupt(); }

// Or use try-with-resources (Java 19+): ExecutorService implements AutoCloseable
```

Explicit ThreadPoolExecutor — the production-grade answer

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
   4,                              // corePoolSize
   8,                              // maximumPoolSize
   60, TimeUnit.SECONDS,           // keep-alive for idle extra threads
   new ArrayBlockingQueue<>(100),  // BOUNDED queue -> backpressure, not OOM
   new CustomThreadFactory("api-worker-"),
   new ThreadPoolExecutor.CallerRunsPolicy()   // rejection policy
);

// Rejection policies:
//   AbortPolicy (default)  -> throws RejectedExecutionException
//   CallerRunsPolicy       -> the submitting thread runs the task (natural throttle)
//   DiscardPolicy          -> silently drops it
//   DiscardOldestPolicy    -> drops the oldest queued task

// Sizing a pool:
//   CPU-bound    -> threads ≈ number of cores
//   I/O-bound    -> threads ≈ cores * (1 + waitTime/computeTime)
int cores = Runtime.getRuntime().availableProcessors();
```

CompletableFuture — composing async work (Java 8+)

```
CompletableFuture<String> cf = CompletableFuture
       .supplyAsync(() -> fetchUser(id), pool)      // runs asynchronously
       .thenApply(user -> user.getName())           // transform the result
       .thenApply(String::toUpperCase)
       .exceptionally(ex -> "FALLBACK");            // recover from errors

String name = cf.get();

// Run several calls in parallel and wait for ALL of them
CompletableFuture<User> u  = CompletableFuture.supplyAsync(() -> getUser(id));
CompletableFuture<List<Order>> o = CompletableFuture.supplyAsync(() -> getOrders(id));
CompletableFuture<Void> all = CompletableFuture.allOf(u, o);
all.join();                                          // wait for both
User user = u.join();
List<Order> orders = o.join();

// wait for the FIRST to complete
CompletableFuture<Object> first = CompletableFuture.anyOf(u, o);

// Chaining dependent async calls (flatMap equivalent)
CompletableFuture<Order> order = CompletableFuture
       .supplyAsync(() -> getUser(id))
       .thenCompose(user -> getLatestOrder(user.getId()));   // returns a Future
```

Concurrent collections and synchronizers in action

```
// ConcurrentHashMap - lock-free reads, per-bin writes
ConcurrentHashMap<String, Integer> hits = new ConcurrentHashMap<>();
hits.merge("page", 1, Integer::sum);        // atomic increment
hits.computeIfAbsent("k", k -> expensiveLoad(k));
// Note:CHM iterators are weakly consistent (no CME), and nulls are forbidden

// BlockingQueue - the backbone of producer/consumer
BlockingQueue<String> q = new ArrayBlockingQueue<>(100);
q.put("job");            // blocks if full
String job = q.take();   // blocks if empty
q.offer("job", 1, TimeUnit.SECONDS);   // returns false on timeout
q.poll(1, TimeUnit.SECONDS);           // returns null on timeout

// CopyOnWriteArrayList - read-heavy, rare writes (listeners, observers)
List<Listener> listeners = new CopyOnWriteArrayList<>();
// Every write copies the whole array -> great reads, expensive writes

// CountDownLatch: start N workers, wait for all to finish
CountDownLatch latch = new CountDownLatch(3);
for (int i = 0; i < 3; i++)
   pool.submit(() -> { try { doWork(); } finally { latch.countDown(); } });
latch.await(10, TimeUnit.SECONDS);   // main resumes when the count hits 0

// CyclicBarrier: N threads wait for each other, then all proceed (reusable)
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("all arrived"));

// Semaphore: allow only N concurrent accesses (connection limiter)
Semaphore sem = new Semaphore(5);
sem.acquire(); try { useResource(); } finally { sem.release(); }
```

SAY IT LIKE THIS IN THE INTERVIEW

"I never create threads directly in production — I submit tasks to an ExecutorService so threads are reused and the pool can be bounded. I construct ThreadPoolExecutor explicitly rather than using the Executors factories, because the default unbounded queue can cause an OOM under load. For results I use Future or CompletableFuture when I need to compose async work, and ConcurrentHashMap or a BlockingQueue instead of synchronizing collections by hand."

Flashcards

Q. Why use a thread pool instead of new Thread()?
A. Thread creation is expensive (~1MB stack each) and unbounded threads exhaust memory. A pool reuses a fixed set of workers, bounds concurrency and adds queueing, monitoring and rejection policies.

Q. Runnable vs Callable?
A. Runnable.run() returns void and can't throw checked exceptions. Callable.call() returns a value and can throw — submit it to an ExecutorService to get a Future.

Q. What is a Future?
A. A handle to an asynchronous computation: get() blocks for the result, cancel() attempts cancellation, isDone() polls. Exceptions surface as ExecutionException.

Q. shutdown vs shutdownNow?
A. shutdown() stops accepting new tasks and lets queued/running ones finish. shutdownNow() attempts to interrupt running tasks and returns the pending queue.

Q. What's wrong with Executors.newCachedThreadPool()?
A. It creates unbounded threads under bursty load — each with its own stack — which can OOM. Prefer a fixed pool backed by a bounded queue.

Q. What is CallerRunsPolicy?
A. A ThreadPoolExecutor rejection policy where the submitting thread runs the rejected task itself — which applies backpressure and slows the producer down.

Q. How do you size a thread pool?
A. CPU-bound: ≈ number of cores. I/O-bound: cores × (1 + wait/compute time). Measure and tune; the queue must stay bounded.

Q. CountDownLatch vs CyclicBarrier?
A. CountDownLatch is a one-shot countdown (N events → waiting threads proceed). CyclicBarrier makes N threads wait for each other and is reusable.

Q. What's a Semaphore used for?
A. Limiting concurrent access to a resource — acquire() takes a permit, release() gives it back. Think connection pools and rate limiting.

Q. BlockingQueue vs ConcurrentLinkedQueue?
A. BlockingQueue blocks on put/take (perfect for producer–consumer, supports bounds). ConcurrentLinkedQueue is non-blocking and unbounded.

Quiz

Q1. What is the main risk of `Executors.newCachedThreadPool()`?

[ ] A. It's too slow
[x] B. It creates unbounded threads under load, which can exhaust memory
[ ] C. It can't run Callables
[ ] D. It reuses only one thread

> **Why:** It has no upper bound and a synchronous handoff queue, so a burst of tasks creates a thread per task. In production, bound both the pool and its queue.

Q2. Which returns a value from an async task?

[ ] A. Runnable
[x] B. Callable
[ ] C. Thread
[ ] D. Executor

> **Why:** Callable<V>.call() returns V and may throw checked exceptions; Runnable returns void.

Q3. What happens if you forget to shut down an ExecutorService?

[ ] A. Nothing, GC cleans it up
[x] B. Its non-daemon worker threads keep the JVM alive — the app may never exit
[ ] C. It throws after 60 seconds
[ ] D. Tasks are silently dropped

Why: Pool threads are non-daemon and stay alive awaiting work. Always call shutdown() (in a finally or @PreDestroy).

Q4. Which is the safest way to increment a counter in ConcurrentHashMap?

[ ] A. map.put(k, map.get(k) + 1)
[x] B. map.merge(k, 1, Integer::sum)
[ ] C. synchronized (map) { map.put(k, map.get(k)+1); }
[ ] D. map.computeIfPresent

Why: merge (and compute/computeIfAbsent) perform the read-modify-write atomically. A manual get-then-put races with other threads.

Q5. What does CompletableFuture.thenCompose do vs thenApply?

[ ] A. They're the same
[x] B. thenApply transforms the value (map); thenCompose flattens a nested future (flatMap)
[ ] C. thenCompose runs synchronously
[ ] D. thenApply handles errors

Why: thenApply gives you CompletableFuture<CompletableFuture<T>> if your function returns a Future; thenCompose flattens that to CompletableFuture<T> — exactly map vs flatMap.

Q6. You need to run 5 independent API calls and wait for all before continuing. Which tool?

[ ] A. CountDownLatch only
[x] B. CompletableFuture.allOf(...).join()
[ ] C. A single thread
[ ] D. Semaphore

Why: allOf returns a future that completes when every given future completes; join() then blocks until they're all done.

8. File I/O & Serialization Core 30 min

Not deeply tested, but cheap marks: the NIO.2 API (`Files`/`Paths`) and the serialization rules (`Serializable`, `serialVersionUID`, `transient`).

Key points

- `java.io` (streams: `FileInputStream`, `BufferedReader`) is the classic API; **NIO.2** (`java.nio.file.Files`, `Paths`) is the modern one — prefer it.
- **Serialization**: converting an object to a byte stream. Implement `Serializable` (a marker interface).
- Declare `static final long serialVersionUID` — without it, any class change invalidates existing serialized data (`InvalidClassException`).
- `transient` fields are **skipped** during serialization (passwords, caches, non-serializable members).
- Static fields are not serialized (they belong to the class, not the object). Deserialization does **not** call the constructor.
- Serialization is considered fragile for persistence — JSON (Jackson) is preferred in modern apps.

Code

NIO.2 — the modern file API (Java 7+)

```
Path path = Paths.get("data", "users.txt");     // platform-independent
Path abs  = path.toAbsolutePath();

// Read
List<String> lines   = Files.readAllLines(path);          // small files
String content       = Files.readString(path);            // Java 11
try (Stream<String> s = Files.lines(path)) {              // large files, lazy
   s.filter(l -> l.contains("ERROR")).forEach(System.out::println);
}
try (BufferedReader br = Files.newBufferedReader(path)) {
   String line; while ((line = br.readLine()) != null) { }
}

// Write
Files.write(path, List.of("a", "b"));
Files.writeString(path, "hello");                          // Java 11
Files.write(path, data, StandardOpenOption.APPEND);

// Inspect and manipulate
Files.exists(path); Files.isDirectory(path); Files.size(path);
Files.copy(src, dst, StandardCopyOption.REPLACE_EXISTING);
Files.move(src, dst);
Files.deleteIfExists(path);
Files.createDirectories(Paths.get("a/b/c"));
Files.walk(Paths.get("src")).filter(p -> p.toString().endsWith(".java")).toList();

// Always close streams - or use try-with-resources (NIO.2 streams are lazily
// loaded, so an unclosed Files.lines() leaks a file handle).
```

Serialization: the rules

```
class User implements Serializable {
   private static final long serialVersionUID = 1L;   // RECOMMENDED

   private String name;
   private transient String password;    // NOT serialized
   private static int count;             // NOT serialized (static)

   // Constructors are NOT called during deserialization
}

// Write
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
   oos.writeObject(user);
}
// Read
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser"))) {
   User restored = (User) ois.readObject();   // ClassNotFoundException possible
   System.out.println(restored.getName());    // "Parvez"
   System.out.println(restored.getPassword());// null - transient was skipped
}

// Customize with writeObject/readObject for encryption or version migration:
private void writeObject(ObjectOutputStream o) throws IOException {
   o.defaultWriteObject();
   o.writeUTF(encrypt(password));
}
```

Output:

```
Parvez
null
```

Why serialization is falling out of favour

```
// Java serialization problems:
//  - Tightly coupled to the Java class (version-fragile, language-specific)
//  - A known attack surface - deserializing untrusted data is dangerous
//    (gadget chains -> RCE). Java 17 added deserialization filters.
//  - Binary, unreadable, hard to diff or debug

// Modern alternative: JSON with Jackson
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(user);        // serialize
User u      = mapper.readValue(json, User.class);     // deserialize
List<User> users = mapper.readValue(json,
       new TypeReference<List<User>>() {});          // generics need TypeReference

// That's exactly what Spring Boot uses for REST by default.
```

SAY IT LIKE THIS IN THE INTERVIEW

"For file work I use NIO.2 — Files and Paths — which is cleaner and throws better exceptions than the old io streams, and I always close streams with try-with-resources. Serialization converts an object to bytes via the Serializable marker; I declare serialVersionUID to keep compatibility across versions and mark sensitive or non-serializable fields transient. In modern apps I'd usually serialize to JSON with Jackson instead."

Flashcards

Q. What is serialVersionUID and why declare it?
A. A version stamp for a Serializable class. If it changes (or is absent and the class structure changes), deserializing old data throws InvalidClassException.

Q. What does transient do?
A. Marks a field to be skipped during serialization — used for passwords, derived data and non-serializable members.

Q. Is a static field serialized?
A. No — static fields belong to the class, not the object. Neither are transient-marked fields.

Q. Is the constructor called during deserialization?
A. No. The object is rebuilt from the byte stream without invoking any constructor.

Q. java.io vs java.nio.file?
A. java.io is the legacy stream API; NIO.2 (java.nio.file.Files/Paths) is modern, supports symbolic links, file attributes, directory walking and better exceptions.

Q. Why is Java serialization considered risky?
A. Deserializing untrusted data can execute attacker-controlled code via gadget chains. Prefer JSON and validate input; use serialization filters if you must.

Q. How do you read a large file without loading it all into memory?
A. Stream it: Files.lines(path) or a BufferedReader inside try-with-resources.

Quiz

Q1. Which field would NOT be serialized?

[ ] A. private String name
[x] B. private transient String token
[ ] C. private int age
[ ] D. private List<String> tags

> **Why:** transient fields are excluded from the serialized form (restored as their default value). Static fields are also excluded.

Q2. What exception is thrown if the class's serialVersionUID doesn't match the serialized data?

[ ] A. ClassNotFoundException
[x] B. InvalidClassException
[ ] C. NotSerializableException
[ ] D. IOException

> **Why:** InvalidClassException (an IOException subclass) signals a version/structure mismatch. NotSerializableException means the class doesn't implement Serializable.

Q3. What happens if you serialize an object with a non-serializable field?

[ ] A. It's silently skipped
[x] B. NotSerializableException at runtime (unless the field is transient)
[ ] C. Compile error
[ ] D. It's serialized as null

> **Why:** Every non-transient field must be serializable, otherwise the write fails at runtime with NotSerializableException.

Q4. Which is the modern way to read all lines of a file?

[ ] A. new BufferedReader(new FileReader(f))
[x] B. Files.readAllLines(Paths.get(f))
[ ] C. System.in
[ ] D. Files.read()

> **Why:** NIO.2's Files utility is concise and handles charset and resource management better. For big files, prefer Files.lines() as a stream.

9. JDBC — Connecting Java to a Database Must-know 45 min

Even in a Spring Boot interview they'll check that you know what happens underneath Hibernate: JDBC, PreparedStatement, transactions. And "why PreparedStatement over Statement" is a guaranteed question because it's about SQL injection.

Key points

- **7 steps**: load the driver (automatic since JDBC 4) → get a Connection → create a Statement → execute a query → process the ResultSet → close everything → handle SQLException.
- **PreparedStatement vs Statement**: precompiled, prevents SQL injection, faster for repeated execution, supports parameter binding and LOBs/setDate.
- **Always use try-with-resources** — leaked connections exhaust the pool fast.
- **Transactions**: `setAutoCommit(false)` → work → `commit()` / `rollback()` in the catch block. Default is auto-commit = true.
- `Statement.executeQuery()` → SELECT (returns ResultSet); `executeUpdate()` → INSERT/UPDATE/DELETE (returns the row count); `execute()` → any.
- **Connection pooling** (HikariCP is the Spring Boot default) reuses physical connections — opening one is expensive.

Code

A complete, correct JDBC query (memorise this shape)

```
String sql = "SELECT id, name, email FROM users WHERE dept = ? AND active = ?";

try (Connection conn = dataSource.getConnection();          // 1. connection
    PreparedStatement ps = conn.prepareStatement(sql)) {   // 2. statement

   ps.setString(1, "Engineering");    // 3. bind parameters (1-indexed!)
   ps.setBoolean(2, true);

   try (ResultSet rs = ps.executeQuery()) {                // 4. execute
       while (rs.next()) {                                 // 5. iterate
           long id    = rs.getLong("id");
           String name = rs.getString("name");
           String email = rs.getString("email");
           System.out.printf("%d %s %s%n", id, name, email);
       }
   }
} catch (SQLException e) {                                  // 6. handle
   throw new DataAccessException("query failed", e);
}   // 7. everything auto-closed by try-with-resources
```

SQL injection — why PreparedStatement matters

```
String userInput = "' OR '1'='1";

// VULNERABLE: string concatenation
String bad = "SELECT * FROM users WHERE name = '" + userInput + "'";
// -> SELECT * FROM users WHERE name = '' OR '1'='1'   -> returns EVERY row!
// Worse: input like "'; DROP TABLE users; --"

// SAFE: parameter binding. The driver sends the SQL and the values separately,
// so the input can never be parsed as code.
String good = "SELECT * FROM users WHERE name = ?";
PreparedStatement ps = conn.prepareStatement(good);
ps.setString(1, userInput);      // treated strictly as DATA
```

Transactions — the classic money-transfer question

```
public void transfer(long fromId, long toId, BigDecimal amount) throws SQLException {
   try (Connection conn = dataSource.getConnection()) {
       conn.setAutoCommit(false);              // 1. begin transaction
       try {
           debit(conn, fromId, amount);        // 2. work
           credit(conn, toId, amount);
           conn.commit();                      // 3. commit on success
       } catch (SQLException e) {
           conn.rollback();                    // 4. undo on ANY failure
           throw e;
       } finally {
           conn.setAutoCommit(true);           // 5. restore for pooled reuse
       }
   }
}
// ACID: Atomicity (all or nothing), Consistency (invariants hold),
// Isolation (concurrent transactions don't interfere), Durability (committed = persisted)

// Isolation levels (weakest -> strongest), set with:
// conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
//   READ_UNCOMMITTED -> dirty reads possible
//   READ_COMMITTED   -> default in Postgres/Oracle; no dirty reads
//   REPEATABLE_READ  -> same row reads give the same values (MySQL default)
//   SERIALIZABLE     -> fully isolated, slowest
```

INSERT with a generated key, and batching

```
// Get the auto-generated primary key back
String sql = "INSERT INTO users(name, email) VALUES (?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql,
                            Statement.RETURN_GENERATED_KEYS)) {
   ps.setString(1, "Parvez");
   ps.setString(2, "p@example.com");
   int rows = ps.executeUpdate();            // number of affected rows
   try (ResultSet keys = ps.getGeneratedKeys()) {
       if (keys.next()) long newId = keys.getLong(1);
   }
}

// Batch insert - one round trip instead of N
try (PreparedStatement ps = conn.prepareStatement(sql)) {
   for (User u : users) {
       ps.setString(1, u.getName());
       ps.setString(2, u.getEmail());
       ps.addBatch();                        // queue it
       if (++i % 500 == 0) ps.executeBatch(); // flush periodically
   }
   ps.executeBatch();                        // flush the remainder
}
```

Connection pooling with HikariCP

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
config.setUsername("root");
config.setPassword("secret");
config.setMaximumPoolSize(10);          // cap concurrent connections
config.setMinimumIdle(2);
config.setConnectionTimeout(30_000);    // fail fast rather than hang
config.setIdleTimeout(600_000);
config.setAutoCommit(false);

HikariDataSource ds = new HikariDataSource(config);
try (Connection c = ds.getConnection()) { /* returns to the pool on close */ }

// Why: opening a TCP connection + authenticating costs ~10-50ms.
// A pool keeps them warm and hands out the same physical connections.
// Spring Boot auto-configures HikariCP - it's the default since Boot 2.
```

SAY IT LIKE THIS IN THE INTERVIEW

"JDBC is the layer under JPA: get a connection from the pool, prepare a statement, bind parameters, execute, process the ResultSet and close everything in try-with-resources. I always use PreparedStatement — it's precompiled, faster on repeat, and critically it prevents SQL injection because the values are sent separately from the SQL. For writes I turn off auto-commit, commit on success and roll back on any exception."

Flashcards

Q. What are the 7 JDBC steps?
A. Register the driver (automatic since JDBC 4), open a Connection, create a Statement/PreparedStatement, execute the query, process the ResultSet, close the resources, handle SQLException.

Q. Statement vs PreparedStatement?
A. PreparedStatement is precompiled, allows parameter binding, prevents SQL injection, handles dates/LOBs cleanly, and is faster when executed repeatedly. Prefer it always.

Q. How does PreparedStatement prevent SQL injection?
A. The SQL is compiled first with placeholders; user values are then sent as data and can never be parsed as SQL syntax.

Q. executeQuery vs executeUpdate vs execute?
A. executeQuery → SELECT, returns a ResultSet. executeUpdate → INSERT/UPDATE/DELETE, returns the affected row count. execute → any statement, returns a boolean indicating whether a ResultSet was produced.

Q. How do you manage a JDBC transaction?
A. setAutoCommit(false), do the work, commit() on success, rollback() in the catch block, and restore auto-commit in a finally.

Q. What is connection pooling and why?
A. A pool (HikariCP) keeps physical DB connections open and reuses them, avoiding the ~10-50ms cost of establishing and authenticating each connection. Spring Boot uses HikariCP by default.

Q. Are JDBC parameters 0-indexed or 1-indexed?
A. 1-indexed — ps.setString(1, value) refers to the first `?` placeholder.

Q. Name the four ACID properties.
A. Atomicity (all-or-nothing), Consistency (invariants preserved), Isolation (transactions don't interfere), Durability (committed data survives crashes).

Q. What are the transaction isolation levels?
A. READ_UNCOMMITTED (dirty reads), READ_COMMITTED (default for Postgres/Oracle), REPEATABLE_READ (MySQL default), SERIALIZABLE (fully isolated, slowest).

Q. How do you get the generated key after an INSERT?
A. Prepare with Statement.RETURN_GENERATED_KEYS, then read ps.getGeneratedKeys().

Quiz

Q1. Why should you use PreparedStatement instead of Statement?

[ ] A. It's the only way to run SELECT
[x] B. It's precompiled, faster on reuse, and prevents SQL injection via parameter binding
[ ] C. Statement is deprecated
[ ] D. It automatically commits transactions

Why: All of the above benefits — and the security one is the headline. Never build SQL by concatenating user input.

Q2. What does executeUpdate() return?

[ ] A. A ResultSet
[x] B. The number of rows affected
[ ] C. A boolean
[ ] D. The generated key

Why: executeUpdate returns an int row count for INSERT/UPDATE/DELETE. Use executeQuery for SELECT and getGeneratedKeys for keys.

Q3. What is the default value of autoCommit, and what does it mean?

[ ] A. false — you must always commit
[x] B. true — each statement is committed immediately as its own transaction
[ ] C. It depends on the driver
[ ] D. null

Why: By default each SQL statement commits as soon as it executes. For multi-statement units of work (like a transfer) you must set autoCommit(false) and manage commit/rollback yourself.

Q4. Which ResultSet method moves the cursor to the next row?

[x] A. rs.next()
[ ] B. rs.moveNext()
[ ] C. rs.hasNext()
[ ] D. rs.forward()

Why: rs.next() returns false when there are no more rows — the standard while (rs.next()) loop.

Q5. Why is a connection pool important?

[ ] A. It makes SQL faster
[x] B. Opening a real DB connection is expensive (TCP + auth), so connections are created once and reused
[ ] C. JDBC requires one
[ ] D. It prevents SQL injection

Why: Pooling avoids per-request connection setup cost and bounds the number of concurrent DB connections, which protects the database.

10. Design Patterns You Must Know Must-know 40 min

For a fresher they'll ask for 2–3 patterns, almost always including Singleton. Learn to write Singleton three ways and explain the reflection/serialization attacks.

Key points

- **Creational**: Singleton, Factory, Builder, (Prototype).
- **Structural**: Adapter, Decorator, Facade, Proxy.
- **Behavioural**: Strategy, Observer, Template Method, Command, Iterator.
- **Singleton**: private constructor + static instance. Must handle reflection, serialization and cloning.
- The **enum singleton** (Joshua Bloch) is the recommended implementation — JVM-guaranteed uniqueness and serialization safety.
- **Builder** shines with many optional parameters (and Lombok's `@Builder` generates it).
- Spring itself is patterns: DI (Factory), `@Transactional` (Proxy), `JdbcTemplate` (Template Method), events (Observer).

Code

Singleton — five implementations, from fragile to bulletproof

```
// 1. Eager: simple, thread-safe, created even if unused
class Eager {
   private static final Eager INSTANCE = new Eager();
   private Eager() { }
   public static Eager getInstance() { return INSTANCE; }
}

// 2. Lazy synchronized: thread-safe but every call pays for the lock
class LazySync {
   private static LazySync instance;
   private LazySync() { }
   public static synchronized LazySync getInstance() {
       if (instance == null) instance = new LazySync();
       return instance;
   }
}

// 3. Double-checked locking: lazy + fast; `volatile` is MANDATORY (Java 5+)
class DCL {
   private static volatile DCL instance;
   private DCL() {
       if (instance != null) throw new IllegalStateException("Already created");
   }
   public static DCL getInstance() {
       if (instance == null) {                      // first check (no lock)
           synchronized (DCL.class) {
               if (instance == null) instance = new DCL();  // second check
           }
       }
       return instance;
   }
}

// 4. Holder idiom (Bill Pugh): lazy, lock-free, thread-safe - class init is atomic
class Holder {
   private Holder() { }
   private static class Inner { static final Holder INSTANCE = new Holder(); }
   public static Holder getInstance() { return Inner.INSTANCE; }
}

// 5. ENUM - the recommended one (Effective Java, Joshua Bloch)
enum EnumSingleton {
   INSTANCE;
   public void doWork() { }
}
// Free serialization safety (no readResolve needed), reflection-proof, one instance.

// Breaking a naive singleton (the follow-up question):
//  - Reflection: setAccessible(true) on the private ctor  -> guard by throwing
//  - Serialization: deserialize creates a NEW object      -> implement readResolve()
//  - Cloning: clone() creates a copy                      -> don't implement Cloneable
protected Object readResolve() { return INSTANCE; }
```

Builder — for objects with many optional fields

```
public class User {
   private final String name;      // required
   private final String email;     // required
   private final int age;          // optional
   private final String phone;     // optional

   private User(Builder b) { this.name = b.name; this.email = b.email;
                             this.age = b.age; this.phone = b.phone; }

   public static class Builder {
       private final String name;
       private final String email;
       private int age; private String phone;

       public Builder(String name, String email) {
           this.name = name; this.email = email;
       }
       public Builder age(int age)         { this.age = age; return this; }
       public Builder phone(String phone)  { this.phone = phone; return this; }
       public User build() {
           if (email == null || !email.contains("@"))
               throw new IllegalStateException("invalid email");
           return new User(this);
       }
   }
}

User u = new User.Builder("Parvez", "p@example.com")
            .age(24)
            .phone("99999")
            .build();
// vs. telescoping constructors: new User("Parvez","p@x.com", 0, null)

// Lombok equivalent: @Builder on the class -> User.builder().name("x").build()
```

Factory, Strategy and Template Method — the three you'll use most

```
// FACTORY: hide the instantiation logic behind a common interface
interface Notification { void send(String msg); }
class EmailNotification implements Notification { public void send(String m){ } }
class SmsNotification   implements Notification { public void send(String m){ } }

class NotificationFactory {
   public static Notification of(String type) {
       return switch (type.toLowerCase()) {
           case "email" -> new EmailNotification();
           case "sms"   -> new SmsNotification();
           default -> throw new IllegalArgumentException("unknown: " + type);
       };
   }
}

// STRATEGY: swap an algorithm at runtime (this is what DI gives you for free)
interface PaymentStrategy { void pay(double amount); }
class Checkout {
   private final PaymentStrategy strategy;      // injected
   Checkout(PaymentStrategy s) { this.strategy = s; }
   void checkout(double amt) { strategy.pay(amt); }
}
new Checkout(new CardPayment()).checkout(100);

// TEMPLATE METHOD: fixed algorithm, subclass supplies the steps
abstract class ReportGenerator {
   public final String generate() {        // final -> the skeleton can't change
       return header() + body() + footer();
   }
   protected abstract String body();
   protected String header() { return "--- Report ---\n"; }
   protected String footer() { return "--- End ---\n"; }
}
```

Patterns Spring uses — great discussion material

```
// Singleton   - Spring beans are singletons by scope (one per container)
// Factory     - BeanFactory / ApplicationContext create beans for you
// Proxy       - @Transactional, @Cacheable, @Async are AOP proxies
// Template    - JdbcTemplate, RestTemplate, RabbitTemplate (boilerplate removed)
// Observer    - ApplicationEventPublisher / @EventListener
// MVC         - DispatcherServlet + @Controller + ViewResolver
// DI/IoC      - the container injects dependencies instead of you new-ing them
// Builder     - RestClient.builder(), ResponseEntity.ok().body(x).build()
// Front Ctrl  - DispatcherServlet handles every request and routes it

// Being able to say "Spring's @Transactional is a proxy-based implementation of
// the Decorator/Proxy pattern" is a strong, concrete answer.
```

SAY IT LIKE THIS IN THE INTERVIEW

"I know the classic GoF patterns by category — creational like Singleton, Factory and Builder; structural like Adapter, Decorator and Proxy; behavioural like Strategy, Observer and Template Method. For Singleton I'd normally use the enum implementation: the JVM guarantees a single instance and it's safe against reflection and serialization. And I can point at where Spring uses each — @Transactional is a proxy, JdbcTemplate is template method, events are observer."

Flashcards

Q. What is the Singleton pattern and how do you implement it safely?
A. One instance per JVM: private constructor + a static accessor. The enum implementation is preferred (serialization- and reflection-safe); alternatives are the holder idiom and double-checked locking with volatile.

Q. How can a Singleton be broken?
A. Reflection (setAccessible on the private constructor), serialization (a new instance on deserialize — fix with readResolve), and cloning (don't implement Cloneable). Multiple classloaders also create one per loader.

Q. Why is volatile needed in double-checked locking?
A. Without it, another thread can see a partially constructed object due to instruction reordering (the reference published before the constructor finished). volatile establishes the happens-before needed.

Q. When is the Builder pattern better than constructors?
A. With many optional parameters — it avoids telescoping constructors, keeps objects immutable, and lets you validate in build().

Q. Factory vs Abstract Factory?
A. Factory creates one type of object based on input (NotificationFactory.of("email")). Abstract Factory is a factory of factories, producing families of related objects.

Q. What is the Strategy pattern used for?
A. Encapsulating interchangeable algorithms behind an interface so you can swap behaviour at runtime — the basis of dependency injection.

Q. Template Method vs Strategy?
A. Template Method uses inheritance: a fixed algorithm in an abstract class with abstract steps. Strategy uses composition: an injected interface. Strategy is usually preferred (more flexible, testable).

Q. Which Spring feature is an example of the Proxy pattern?
A. @Transactional (and @Cacheable, @Async) — Spring wraps the bean in an AOP proxy that adds behaviour around the method call.

Q. What is the DAO pattern?
A. Data Access Object — an interface + implementation that hides persistence details (SQL/JPA) from the service layer, so the business logic is decoupled from the database.

Quiz

Q1. Which Singleton implementation is recommended in Effective Java?

[ ] A. Lazy synchronized
[ ] B. Double-checked locking
[x] C. Enum singleton

[ ] D. Public static field

> **Why:** The enum singleton is concise, provides free serialization safety, and the JVM guarantees a single instance — no readResolve or reflection guards needed.

Q2. Why is `volatile` required in double-checked locking?

[ ] A. For performance
[x] B. To prevent another thread from seeing a partially constructed instance due to reordering
[ ] C. It isn't required
[ ] D. To make the lock reentrant

> **Why:** Since Java 5, volatile's happens-before guarantee makes the safe publication work. Without it, DCL is broken.

Q3. Which pattern would you use to create objects of related families without specifying concrete classes?

[ ] A. Singleton
[x] B. Abstract Factory
[ ] C. Observer
[ ] D. Adapter

> **Why:** Abstract Factory produces families of related objects (e.g. a UI kit factory returning matching Button + Checkbox for a given OS theme).

Q4. Which pattern removes the need for if/else chains when selecting an algorithm?

[x] A. Strategy
[ ] B. Singleton
[ ] C. Facade
[ ] D. Decorator

> **Why:** Strategy encapsulates each algorithm behind a common interface and lets you inject the one you want — replacing conditional branching with polymorphism.

Q5. JdbcTemplate is an example of which pattern?

[ ] A. Singleton
[x] B. Template Method
[ ] C. Observer
[ ] D. Builder

> **Why:** It fixes the algorithm (get connection, create statement, execute, handle exceptions, close) and lets you supply just the varying part (row mapping / SQL).